UNIX SHELL PROGRAMMING

(24BECSE405)

LAB MANUAL

DEPT OF CSE

SAPTHAGIRI NPS UNIVERSITY

Exercise on working with Directories

1. **Create a directory structure as follows**

```
SNPSU
    SoET
        CSE
        AIML
        AIDS
        DS
    SoAS
        MBA
        MCA
    SoMS
```

Create a directory SNPSU
Without entering the SNPSU directory create the subdirectories SoET, SoAS, SoMS within it.
Change the working directory to SoET
Display the current working directory
Now within the SoET directory create a CSE directory containing two subdirectories II and IV using a single command
Repeat above for AIML, AIDS and DS
Move back to the parent directory of SNPSU directory
List the directory structure recursively
using a single command move to the IV directory of CSE
move to its parent directory
Go to the II folder in DS directory
Using a single command switch to the IV directory of AIML
go back to AIML directory and delete II directory
Go to the SNPSU directory and delete AIML directory
Now move to the II directory within CSE directory
change the directory to the user's home directory
Switch back to the previous working directory

commands used: pwd, mkdir, cd, rmdir

Create a directory SNPS
mkdir SNPSU

Without entering the SNPSU directory create the subdirectories SoET, SoAS, SoMS within it.
mkdir SNPSU/SoET SNPSU/SoAS SNPSU/SoMS

Change the working directory to SoET
cd SNPSU/SoET

Display the current working directory
pwd

Now within the SoET directory create a CSE directory containing two subdirectories II and IV using a single command
mkdir -p CSE/II CSE/IV

Repeat above for AIML, AIDS and DS
mkdir -p AIML/II AIML/IV
mkdir -p AIDS/II AIDS/IV
mkdir -p DS/II DS/IV

Move back to the parent directory of SNPSU directory
cd ..

List the directory structure recursively
tree

using a single command move to the IV directory of CSE
cd SNPSU/SoET/AIML/IV

Go to the II folder in DS directory
cd ../../DS

Using a single command switch to the IV directory of AIML
cd ~-

go back to AIML directory and delete II directory
cd ..
rmdir II

Go to the SNPSU directory and delete AIML directory
cd ..
rm -rf AIML

Now move to the II directory within CSE directory
cd CSE/II

change the directory to the users home directory
cd or cd ~

Switch back to the previous working directory
cd ~-

2. Exercise on working with File Manipulation commands

Inside the IV directory under CSE, create a file students.txt using cat command with few student names in it
Also create a empty file faculty.txt
enter few faculty names using the nano editor
display contents of students using the following commands – cat, tac and rev
display line numbers along with file content
create a copy of students.txt by the name vidhyarthi.txt
copy the file students.txt to IV directory under DS directory
move the faculty.txt to IV directory under AIDS directory
delete the file vidhyarthi.txt
create multiple empty files inside the II directory of CSE using a single command
copy all these files to a newly created subdirectory using a single command
delete all files beginning with the letter a
move all files with a .txt extension to a separate folder
Create a text file, a directory, a C program with its executable and a hidden file
display the file type for each of these files
display the files using ls
display hidden files
display file attributes
Change the file permissions using chmod
Change ownership using chown and chgrp

Inside the IV directory under CSE, create a file students.txt using cat command with few student names in it

```
$cd SoET/CSE/IV
$cat > students.txt
Tony Adams
Rakesh Kumar
Javed Ali
Seeta Kumari
Manpreet Kaur
Ctrl+z
```

Also create a empty file faculty.txt
enter few faculty names using the nano editor

```
$nano faculty.txt
Rakesh Gowda
Anand Kanade
Alekhya
Suhas

Press Ctrl+ x to save and exit
Then press y to confirm
```

display contents of students using the following commands – cat, tac and rev

```
$ cat students.txt
$ tac students.txt
```

$ rev students.txt

display line numbers along with file content
$ cat -n students.txt

create a copy of students.txt by the name vidhyarthi.txt
$ cp students.txt vidhyarthi.txt

copy the file students.txt to IV directory under DS directory
$ cp students.txt ../../DS/IV

move the faculty.txt to IV directory under AIDS directory
$ mv faculty.txt ../../AIDS/IV

delete the file vidhyarthi.txt
$ rm vidhyarthi.txt

create multiple empty files inside the II directory of CSE using a single command
$ cd ../II
$ touch 1.txt 2.txt 3.txt

copy all these files to a newly created subdirectory using a single command
$ mkdir DEST
$ cp *.txt DEST

delete all files beginning with the letter a
$ rm a*

move all files with a .txt extension to a separate folder
$ mv *.txt DEST

Create a text file, a directory, a C program with its executable and a hidden file display the file type for each of these files
$ file *

display the files using ls
$ ls

display hidden files
$ ls -a

display file attributes
$ ls -l

Change the file permissions using chmod
$ chmod a+x file.txt
$ chmod 755 file.txt

Change ownership using chown and chgrp
$ chown file <username>
$ chgrp file <username>:<group>

3. Common utilities

Display date
Display date in mm/dd/yy and dd:mm:yy formats
Display current time only
write a command to create a file such that the filename also contains the date of creation
display calendar for the current month only
display calendar for a specified year
display calendar in alternative layout for a specified year
display users logged into the system
change password
echo command
manual pages
clear screen
system details

Common utilities

Display date
Date

Display date in mm/dd/yy and dd:mm:yy formats
date +%D
date +%d:%m:%y

Display current time only
date +%R
date +%H:%M:%S

display calendar for the current month only
cal

display calendar for a specified year
cal 2026

display calendar in alternative layout for a specified year
write a command to create a file such that the filename also contains the date of creation
touch myfile_`date +%d_%m_%y`.txt

display users logged into the system
who
whoami

change password
passwd

echo command
echo Hi there
echo This is not printed the same
echo "This is printed as it is"
echo "Free Software is the Future \n And the Future is ours"

echo -e "Free Software is the Future \n And the Future is ours"

manual pages
man ls
man cal
man man

clear screen
clear
Ctrl L

system details
uname
uname -a
lsb_release -a
uptime

4. **Horizontal Partitioning**
   Display first N lines of a file
   Display last N lines of a file
   Display skipping first N lines of a file
   Display lines in a range M to N

   Vertical Partitioning
   illustrate column wise and field wise partitioning using cut command
   merge contents column wise from multiple files

   sort file contents by alphabetical order

   Find files matching a criteria
   find by name, file type, permission, access time, modification time, size

Horizontal Partitioning

Display first N lines of a file
head -n 10 oliver.txt

Display last N lines of a file
tail -10 oliver.txt

Display skipping first N lines of a file
tail +10 oliver.txt

Display lines in a range M to N
head -n 20 oliver.txt | tail +9
cat -n oliver.txt | head -20 | tail +9

Vertical Partitioning
illustrate column wise and field wise partitioning using cut command

cut -c61-63,57-59 5sem.txt

cut -f1 5tab.txt
cut -f1,3-5 5tab.txt
cut -f1,3-5 -d"," 5sem.csv

merge contents column wise from multiple files
paste fileeven fileodd
paste fileeven fileodd fileeven
paste -d"#" fileeven fileodd

sort file contents by alphabetical order
sort random.txt

Find files matching a criteria
find by name, file type, permission, access time, modification time, size
find files based on name
find ~ -name "myFirst.*"
find ~ -maxdepth 1 -name "myFirst.*"

Find files by type (d f c b l s)
find Downloads -type d
find Downloads -type d -name "D*"

find .c files and display with custom message
find . -type f -name "*.c" -exec echo "I found {}" \;

Find files by permission
find . -type f -perm 0644 -print
find . -type d -perm 0755 -print
find . -perm /a=x

Find files by access time
find . -amin -5
find . -mmin +25

find files by size
find . -size +10M
find . -size +1M
find . -size -1M

5. **GREP Exercises**

Search for a given word in a file along with its line number.
Perform a case insensitive search for a given word in a file and print the number of occurences.
Display lines that do not contain a given word
Display lines beginning with a Capital letter and ending with!
Count number of blank lines in the file
Display content of the file without blank lines
Select the lines that have atleast two digits without any other character in between
Select the lines from the file that do not start with A to G
Print names of all files containing a given word
Find out whether user2 is currently logged into the system
Check whether a given process is currently executing or not if yes obtain its process id

Grep: global regular expression print

Search for a given word in a file along with its line number.
grep -n elephant 1.txt

Perform a case insensitive search for a given word in a file and print the number of occurences.
grep -c -i the 1.txt

Display lines that do not contain a given word
grep -v -i 'the' 1.txt

Display lines beginning with a Capital letter and ending with !
grep -E -n '^[A-Z].*!.$' oliver.txt

Count number of blank lines in the file
grep -E -c '^$' testFile

Display content of the file without blank lines
grep -E -v '^$' testFile

Select the lines that have atleast two digits without any other character in between
grep -E '[0-9][0-9]' testFile

Select the lines from the file that do not start with A to G
egrep '^[^A-G]' testFile

Print names of all files containing a given word
grep -l 'elephant' *

Find out whether user2 is currently logged into the system
who | grep 'user2'

Check whether a given process is currently executing or not if yes obtain its process id
ps -e | grep gedit
ps -e | grep vlc

6. Shell Programming Control Structures

Illustrate various shell programming control structures that can be used in a shell script
Desision Control Structures
Looping Control Structures
Jump Control Structures
Function Control Structures

Shell Programming Control Structures

Program to illustrate if Statement

```
echo "1.1 if-then statement:"
if [ 5 -gt 3 ]; then
   echo -e "  5 is greater than 3"
fi
```

Program to illustrate if_else Statement

```
echo "\n1.2 if-then-else statement:"
echo "Enter a number"
read number
if [ $number -eq 10 ]; then
   echo -e " Number is 10"
else
   echo -e " Number is not 10"
fi
```

Program to illustrate if-elif-else Statement

```
echo "\n1.3 if-elif-else statement:"
echo "Ente score [0-100]"
read score
if [ $score -ge 90 ]; then
   echo "  Grade: A"
elif [ $score -ge 80 ]; then
   echo "  Grade: B"
elif [ $score -ge 70 ]; then
   echo "  Grade: C"
else
   echo "  Grade: F"
fi
```

Program to illustrate Case Statement

```
echo "\n1.5 Case statement:"
day=$(date +%A)
case $day in
   Monday)
      echo "  Today is Monday - Start of work week"
      ;;
   Tuesday|Wednesday|Thursday)
      echo "  Today is $day - Middle of work week"
      ;;
   Friday)
```

```
        echo "  Today is Friday - TGIF!"
        ;;
    Saturday|Sunday)
        echo "  Today is $day - Weekend!"
        ;;
    *)
        echo "  Invalid day"
        ;;
esac
```

Program to illustrate JUMP CONTROL Statement

```
echo "3. JUMP CONTROL STATEMENTS"

# break statement
echo "3.1 Break statement:"
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        echo "  Breaking at $i"
        break
    fi
    echo "  Number: $i"
done

# continue statement
echo "\n3.2 Continue statement:"
for i in {1..5}; do
    if [ $i -eq 3 ]; then
        echo "  Skipping $i"
        continue
    fi
    echo "  Number: $i"
done
```

Program to illustrate LOOPING CONSTRUCTS Statement

```
echo "2. LOOPING CONSTRUCTS"
# for loop with list
echo "2.1 For loop with list:"
for fruit in apple banana orange mango; do
    echo "  Fruit: $fruit"
done

# for loop with range
echo "2.2 For loop with range (C-style):"
for (( i=1; i<=5; i++ )); do
    echo "  Iteration: $i"
done

# for loop with array
echo "2.3 For loop with array:"
colors=("Red" "Green" "Blue" "Yellow")
for color in "${colors[@]}"; do
```

```
    echo "  Color: $color"
done

# while loop
echo "2.4 While loop:"
counter=1
while [ $counter -le 5 ]; do
    echo "  Counter: $counter"
    ((counter++))
done

# until loop
echo "2.5 Until loop:"
counter=1
until [ $counter -gt 5 ]; do
    echo "  Until counter: $counter"
    ((counter++))
done

# nested loops
echo "2.6 Nested loops (multiplication table):"
for i in 1 2 3; do
    for j in 1 2 3; do
        echo -n "  $((i * j)) "
    done
    echo ""
done
```

Program to illustrate FUNCTION CONTROL Statement

```
echo "4. FUNCTION CONTROL STRUCTURES"
# Simple function
greet() {
    echo "  Hello, $1!"
}
echo "4.1 Function with parameter:"
greet "World"
greet "Everyone"

# Function with return value
is_even() {
    if [ $(($1 % 2)) -eq 0 ]; then
        return 0  # true
    else
        return 1  # false
    fi
}
echo "\n4.2 Function with return value:"
echo "Enter a number"
read number
if is_even $number; then
    echo "  $number is even"
```

```
else
   echo "  $number is odd"
fi
```

Illustrate various shell programming control structures that can be used in a shell script

```
echo "1.1 if-then statement:"
if [ 5 -gt 3 ]; then
   echo -e "  5 is greater than 3"
fi
echo "\n1.2 if-then-else statement:"
read number
if [ $number -eq 10 ]; then
   echo -e " Number is 10"
else
   echo -e " Number is not 10"
fi
```

7. **Write a shell Script that checks a directory for files ending with a .c extension, if present create a backup of those files into a timestamped archive.**

```
#!/usr/bin/env bash

# backup_c_files.sh - Backup all .c files from a given directory into a timestamped archive.

# --- Function to display usage ---
usage() {
   echo "Usage: $0 <source_directory>"
   echo "  <source_directory>   Directory containing .c files to back up."
   echo "The backup archive will be created in the current directory with a name like:"
   echo "  c_backup_YYYYMMDD_HHMMSS.tar.gz"
   exit 1
}

# --- Check argument ---
if [ $# -ne 1 ]; then
   usage
fi

src_dir="$1"

# --- Validate source directory ---
if [ ! -d "$src_dir" ]; then
   echo "Error: Source directory '$src_dir' does not exist or is not a directory." >&2
   exit 1
fi

timestamp=$(date +%Y%m%d_%H%M%S)
archive_name="c_backup_${timestamp}.tar.gz"
count=`find $src_dir -maxdepth 1 -name "*.c" -type f | wc -l`

if [[ $count -eq 0 ]]; then
   echo "No .c files found in '$src_dir'. Nothing to back up."
   exit 0
fi

cd $src_dir
tar -czf "$archive_name" *.c
mv $archive_name ..
echo "Backup successfully created: $archive_name"
```

8. **Exercise on Command line Networking:** Demonstrate the commands
ifconfig
ip
ping
ssh
scp

Display the network interface and ip address of your system

Check if a remote host is online

Perform a remote login using Secure Shell(ssh)

Copy files from/to a remote host using Secure Copy(scp)

Command line networking

view system details
network interface details
ifconfig
ip addr show

ping
Ping verifies if a device is reachable, checks network responsiveness, and helps diagnose connectivity issues like packet loss, high latency, or unreachable hosts.
ssh
ssh -l moodle moodle.sit.ac.in
ssh -l snpsu 192.168.16.59
scp random.txt moodle@moodle.sit.ac.in:/home/moodle/recv
scp moodle@moodle.sit.ac.in:/home/moodle/moodle.png .
scp -r folder moodle@moodle.sit.ac.in:/home/moodle/recv

9. **Exercises on sed**
   Double spacing a file

   Replace the occurence of a specific word by an other word in a given file

   sed script removes blank lines that immediately follow lines beginning with an uppercase letter.

   Exercises on sed

   Double spacing a file
   sed -f insertBlankLines.sed random.txt

   Replace the occurence of a specific word by an other word in a given file
   Address s / Pattern / Replacement String / Flag(s)
   sed '/good/s/good/great/' random.txt
   sed '/good/s//great/' random.txt

   sed script removes blank lines that immediately follow lines beginning with an uppercase letter.

   /^[A-Z]/ – Matches any line that starts with an uppercase letter (A through Z).
   { ... } – Groups commands to execute when the pattern matches.
   Inside the group:

   n – Prints the current line (the uppercase line) if auto-print is on (default), then reads the next line into the pattern space, replacing it.

   /^$/d – Checks if that newly read line is empty. If it is, it deletes it (so it won't be printed).

10. Exercises on awk

Rearrange a phone number list file in the order phone number, first name and last name

For each word in the wordlist stored in a file display the word along with its frequency

Illustrate mathematical functions in awk

Exercises on awk

Rearrange a phone number list file in the order phone number, first name and last name

awk -f 18phoneList.awk phoneBook.dat

For each word in the wordlist stored in a file display the word along with its frequency

awk -f 13WordCount.awk text2.dat

Illustrate mathematical functions in awk

awk -f 22mathFunc.awk